

SQL Injection UNION Attack: Retrieving Data from Other Tables

1. Vulnerability Overview

- **Vulnerability Type:** SQL Injection — UNION Attack / Data Extraction
- **Security Impact:** Unauthorized Database Access
- **Primary Objective:** Extract sensitive user information, such as usernames and passwords, from a separate database table.

2. Vulnerability Description

Once the number of columns and compatible data types of the original query have been identified, a UNION-based SQL injection may be used to combine the application's legitimate query with an attacker-controlled query.

The injected query can potentially reference other database tables, such as a `users` table, causing sensitive records to be included in the application's normal response.

3. Attack Vector & Exploitation Methodology

The assessment first determines the structure of the vulnerable SQL query.

After establishing the required column count and compatible data types, the tester evaluates whether a UNION operation can be used to retrieve data from other database structures.

If successful, records from unauthorized tables may be returned through the application's HTTP response.

4. Security Impact

Successful exploitation may result in:

- Unauthorized access to database records.
- Exposure of usernames and password-related information.
- Disclosure of sensitive application data.
- Potential account compromise if exposed credentials are usable.
- Further compromise of application functionality through chained attacks.

5. Root Cause

The vulnerability results from the application's failure to properly parameterize user-controlled input before incorporating it into SQL queries.

Insufficient database privilege separation may further increase the amount of information accessible through the compromised query.

6. Remediation

- Implement **parameterized queries / prepared statements** throughout the application.
- Apply the **Principle of Least Privilege** to application database accounts.
- Ensure application accounts cannot access tables or operations they do not require.
- Avoid exposing raw database errors to users.
- Implement appropriate server-side input validation as a defense-in-depth measure.

7. Security Recommendation

Parameterized database access should be the primary control against SQL injection, while database privilege restrictions should limit the potential impact if another vulnerability is exploited.

Author: Moe Htet Ar Kar (Phoe Cho)